

INSTITUTO INFNET

SISTEMAS ROBÓTICOS

VISÃO COMPUTACIONAL COM OPENCV (DR2)

PROFESSOR: VITOR AMADEU

GABRIEL CRUZ FERREIRA

AT — ASSESSMENT

Relatório técnico: pipeline de percepção visual — calibração de câmera, realidade aumentada, classificação e detecção profunda, rastreamento com ID persistente e segmentação semântica

Rio de Janeiro, 23 de setembro de 2026

Sumário

Seção	Conteúdo
1	Ambiente, instalação e execução
2	Exercício 1 — Calibração de câmera e realidade aumentada
3	Exercício 2 — Classificação com OpenCV DNN e pipeline integrado
4	Exercício 3 — Detecção com YOLO e SSD e rastreamento com ID
5	Exercício 4A — Segmentação semântica comparada à segmentação por cor
6	Exercício 4B — Relatório integrativo
7	Conclusão e repositório

Todas as métricas deste documento foram medidas na máquina de desenvolvimento (Windows 11, Python 3.13.12, CPU, sem GPU) pelos scripts entregues. As saídas integrais de terminal estão em `evidencias/*_terminal.txt`.

1. Ambiente, instalação e execução

1.1 Instalação

```
cd AT
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements.txt
.venv/Scripts/python.exe -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

Pacote	Versão usada nas medições
opencv-python	4.14.0.94
numpy	2.5.3
tensorflow	2.21.0
torch / torchvision	2.14.0+cpu / 0.29.0+cpu
matplotlib / psutil	3.11.2 / 7.2.2
notebook / nbconvert	7.6.3 / 7.17.1

1.2 Execução

A ordem importa: `ex1a.py` gera a calibração consumida pelo `ex1b.py` e pelo notebook; `ex2a.py` gera o modelo `.tflite` usado no notebook.

```
# Preparação (baixa modelos e imagens; idempotente)
.venv/Scripts/python.exe preparar_dados_ex2.py
.venv/Scripts/python.exe preparar_modelos_ex3.py
.venv/Scripts/python.exe preparar_dados_ex4.py

# Exercícios
.venv/Scripts/python.exe ex1a.py
.venv/Scripts/python.exe ex1b.py
.venv/Scripts/python.exe ex2a.py
.venv/Scripts/python.exe -m nbconvert --to notebook --execute --inplace ex2b.ipynb
.venv/Scripts/python.exe ex3a.py
.venv/Scripts/python.exe ex3b.py
.venv/Scripts/python.exe ex4a.py
.venv/Scripts/python.exe ex4b.py
```

Execução completa: cerca de 9 minutos nesta máquina.

1.3 Duas restrições de ambiente que precisaram de tratamento

O OpenCV 5 não serve para este trabalho. A versão 5.0 removeu `cv2.HOGDescriptor`, `cv2.CascadeClassifier` e os rastreadores do módulo principal, usados no Exercício 2B. Verificado nesta máquina: com a 5.0.0 instalada globalmente, os três atributos não existem. Por isso o `requirements.txt` fixa `opencv-python<5` e o ambiente do AT é separado do Python global.

Acento no caminho quebra a entrada e saída do OpenCV. A pasta do trabalho contém acento (*Sistemas Robóticos*), e nesse caso `cv2.imread`, `cv2.imwrite`, `cv2.VideoCapture`, `cv2.VideoWriter` e os carregadores `cv2.dnn.readNetFrom*` falham — quase sempre em silêncio (`imwrite` devolve `False`; `imread` devolve `None`). O módulo `common.py` centraliza a solução: `imread_u` e `imwrite_u` fazem a I/O em Python com `np.fromfile` e `cv2.imdecode/imencode`, e `caminho_ascii` copia modelos e vídeos para uma pasta temporária sem acento antes de abrir. Em pastas sem acento o comportamento é idêntico, sem cópia.

2. Exercício 1 — Calibração de câmera e realidade aumentada

Competências 1.2, 1.3 e 4.1 · Arquivos: [camera_virtual.py](#), [ex1a.py](#), [ex1b.py](#)

2.1 O que o enunciado pedia

Item A: capturar ao menos 15 imagens de um tabuleiro em ângulos e distâncias variados, obter a matriz K , os 5 coeficientes de distorção e os erros de reprojeção, aplicar `cv2.undistort` e exibir original e corrigida lado a lado. Item B: estimar a pose com `cv2.solvePnP`, projetar um cubo 3D de aresta igual a um quadrado com `cv2.projectPoints`, desenhar as arestas com cores distintas por face e exibir rotação e translação a cada frame.

2.2 Como resolvemos

Seguimos a abordagem de **câmera virtual** adotada na disciplina: as imagens do tabuleiro são sintetizadas por uma câmera de parâmetros conhecidos. A vantagem é decisiva para avaliação — com webcam só se conhece o erro de reprojeção, enquanto aqui existe o valor verdadeiro de K e é possível medir o **erro real** da calibração. O [ex1b.py](#) tem modo `--webcam`, que demonstra que o pipeline é o mesmo com câmera física.

Três decisões nossas, além do material de aula:

1. **Distorção de lente não nula.** A câmera virtual de referência tem distorção zerada, e nesse caso o `undistort` devolve uma imagem idêntica à original: o painel exigido no item A não mostraria efeito algum. Injetamos uma distorção barril moderada ($k_1 = -0,26$), típica de webcam, aplicada por remapeamento inverso com `cv2.undistortPoints` sobre a grade completa de pixels.
2. **Plano com grade de linhas retas.** A distorção radial só é perceptível longe do centro óptico, e o tabuleiro ocupa a região central. Colocá-lo no meio de um plano quadriculado torna a curvatura visível na borda e a correção verificável a olho nu. Todo o plano está em $Z = 0$, então a geometria continua consistente.
3. **24 imagens, sendo 6 com o padrão na periferia do quadro.** O mínimo pedido é 15. Acrescentamos poses que levam o tabuleiro aos quatro cantos, mais uma bem próxima e uma distante.

2.3 Evidências

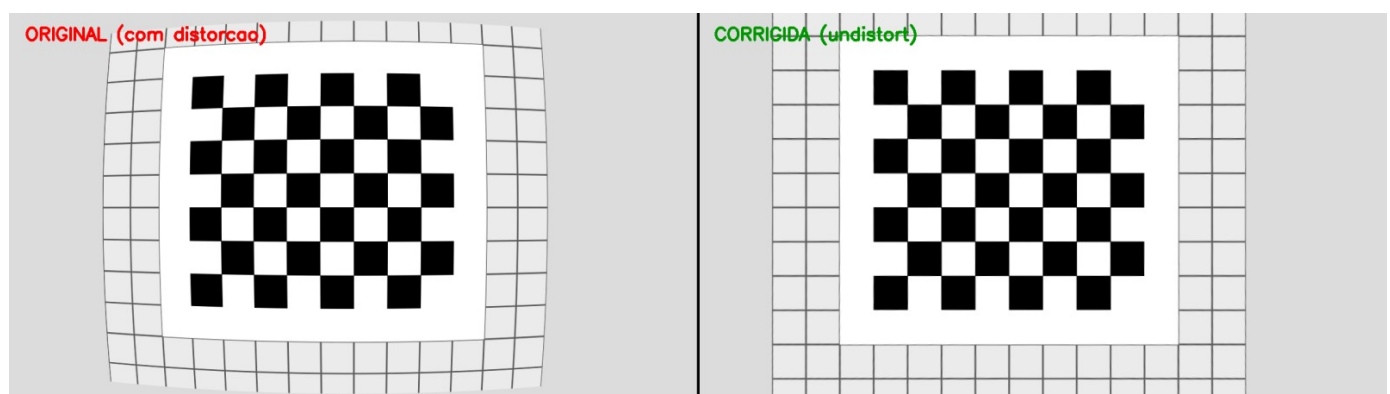


Figura 1 — Painel exigido no item A: à esquerda a imagem como sai da câmera, com a grade curvada pela distorção barril; à direita, depois do `cv2.undistort`, com as linhas retas restauradas

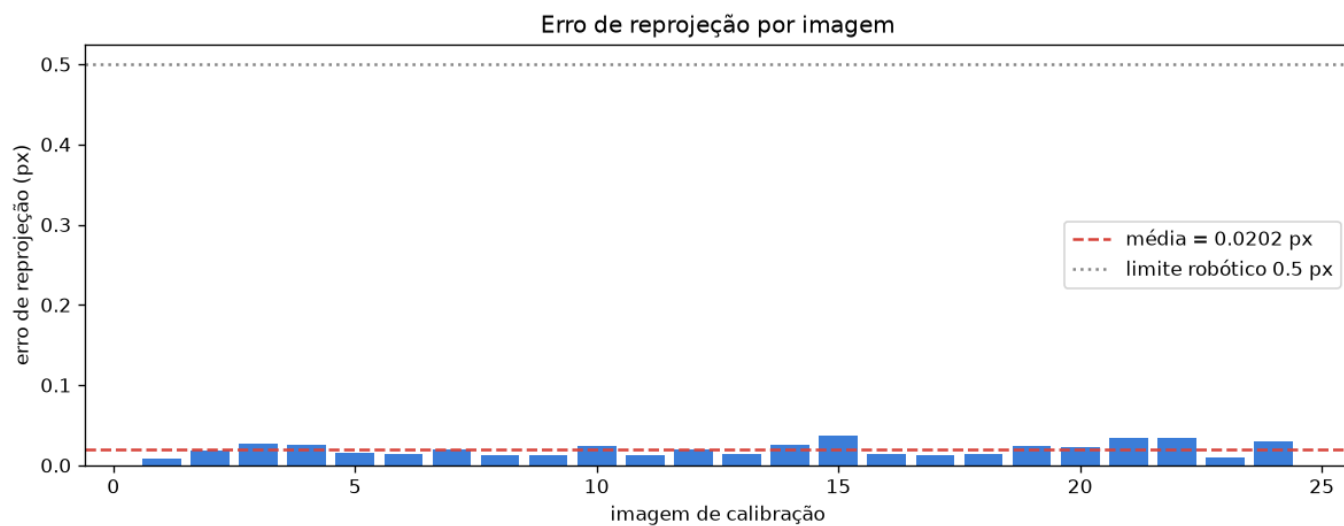


Figura 2 — Erro de reprojeção por imagem, com a média medida e o limite de 0,5 px considerado aceitável para robótica

verde: 18 poses centrais vermelho: poses periféricas

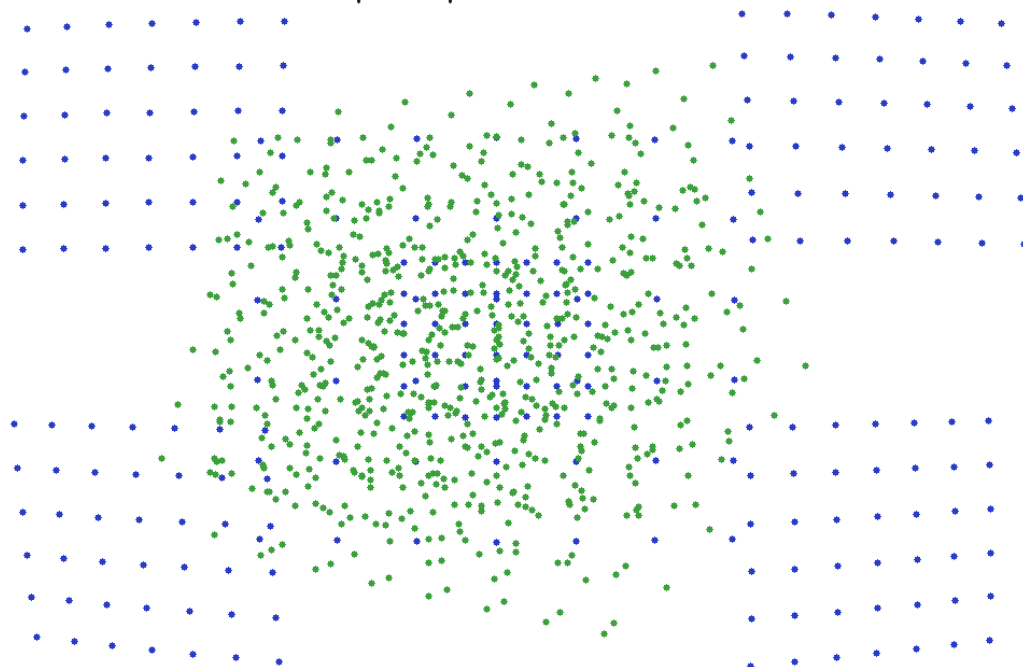


Figura 3 — Cobertura do quadro pelos cantos detectados: em verde as 18 poses centrais do material de aula, em vermelho as 6 poses periféricas acrescentadas

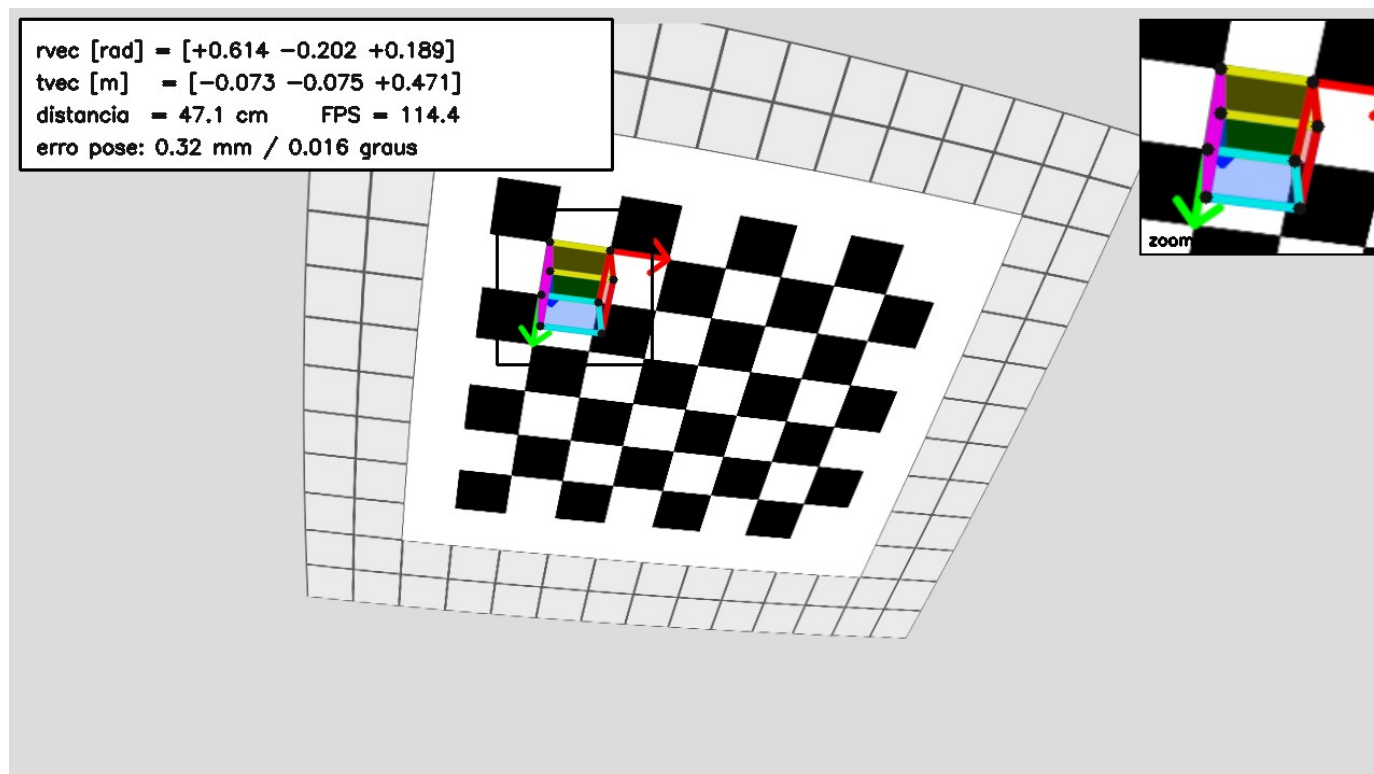


Figura 4 — Item B: cubo de aresta igual a um quadrado (30 mm) projetado sobre o canto de origem, com uma cor por face, eixos do padrão e recorte ampliado. O painel mostra rvec, tvec, distância e erro de pose do frame

2.4 Resultados medidos

Métrica	Valor
Imagens com o padrão detectado	24 de 24
fx estimado (verdadeiro 900,00)	900,70 px — erro de 0,078%
fy estimado (verdadeiro 910,00)	910,68 px — erro de 0,075%
Coeficiente k1 (verdadeiro -0,260000)	-0,260050
Erro médio de reprojeção	0,0202 px
Erro de translação da pose (item B)	0,31 mm em média, desvio de 0,03 mm
Erro de rotação da pose	0,028° em média
Padrão detectado no vídeo	90 de 90 frames
Custo por frame	detecção 6,03 ms · solvePnP 0,25 ms · projeção 0,02 ms

2.5 O que descobrimos

O erro de reprojeção, sozinho, engana. Calibramos duas vezes para medir o efeito da cobertura do campo de visão:

Conjunto	Erro de reprojeção	Erro do modelo de distorção na borda
18 poses centrais	0,0182 px	27,5 px
+ 6 poses periféricas (24)	0,0202 px	0,24 px

O erro de reprojeção ficou ligeiramente **pior** com as 24 imagens, e mesmo assim a calibração ficou cerca de 100 vezes melhor na periferia do quadro. A razão é simples: o erro de reprojeção só mede o ajuste onde

o padrão apareceu. Em um robô, isso é a diferença entre medir bem no centro da imagem e medir bem em toda ela, onde costumam aparecer os obstáculos laterais.

Coefficiente não se avalia isolado. k_1 , k_2 e k_3 são fortemente correlacionados, e combinações diferentes produzem quase o mesmo deslocamento na região observada. Por isso medimos o erro do **modelo** — de quanto os dois modelos discordam ao mapear cada pixel do quadro — em vez de comparar coeficiente a coeficiente.

A detecção do padrão domina o custo (95,6% do frame), enquanto `solvePnP` e `projectPoints` são desprezíveis. Em robótica, isso indica que trocar o tabuleiro por um marcador mais leve, como ArUco ou AprilTag, é o caminho para ganhar FPS — e não otimizar a estimativa de pose.

3. Exercício 2 — Classificação com OpenCV DNN e pipeline integrado

Competências 2.4, 3.3 e 4.2 · Arquivos: `preparar_dados_ex2.py`, `ex2a.py`, `pipeline_ex2b.py`, `ex2b.ipynb`

3.1 Item A — o que o enunciado pedia

Carregar um modelo pré-treinado pelo módulo DNN do OpenCV, processar ao menos 10 imagens de categorias distintas exibindo o top-3, medir a latência contra a mesma rede no Keras e montar uma tabela com latência, memória e acurácia top-1.

3.2 Como resolvemos

A rede escolhida foi a MobileNetV2, porque é a única entre as sugeridas que existe nativamente nos dois lados. GoogLeNet e SqueezeNet só estão disponíveis em Caffe, e o Keras não os traz; comparar GoogLeNet-Caffe com InceptionV3-Keras mediria a diferença entre duas **redes**, não entre dois **backends**. O caminho de conversão é o do material de aula: Keras → TensorFlow Lite → `cv2.dnn.readNetFromTFLite`.

Três cuidados para que a comparação fosse justa:

1. **Fotos reais em vez de sintéticas.** As imagens do material de aula são retângulos e círculos coloridos, úteis para testar o fluxo de arquivos. Mas o enunciado exige acurácia top-1, e sobre figuras geométricas ela é sempre zero. Baixamos 12 fotografias reais, uma por categoria do ImageNet, predominantemente urbanas, com o código da classe embutido no nome do arquivo — o que dá gabarito exato, sem comparação frágil por texto.
2. **Memória medida em processos separados.** Medir os dois backends no mesmo processo atribui ao OpenCV a memória que o TensorFlow já havia alocado. O `ex2a.py` se re-executa em subprocesso para isolar cada um.
3. **Três formas de chamar o Keras.** A latência depende de como a rede é invocada: 116,80 ms com `model(x)`, 65,43 ms com `predict(x)` e 13,87 ms com `tf.function`. Usar a pior variante inflaria artificialmente a vantagem do OpenCV, então a tabela final usa a **melhor** do Keras.

3.3 Evidências

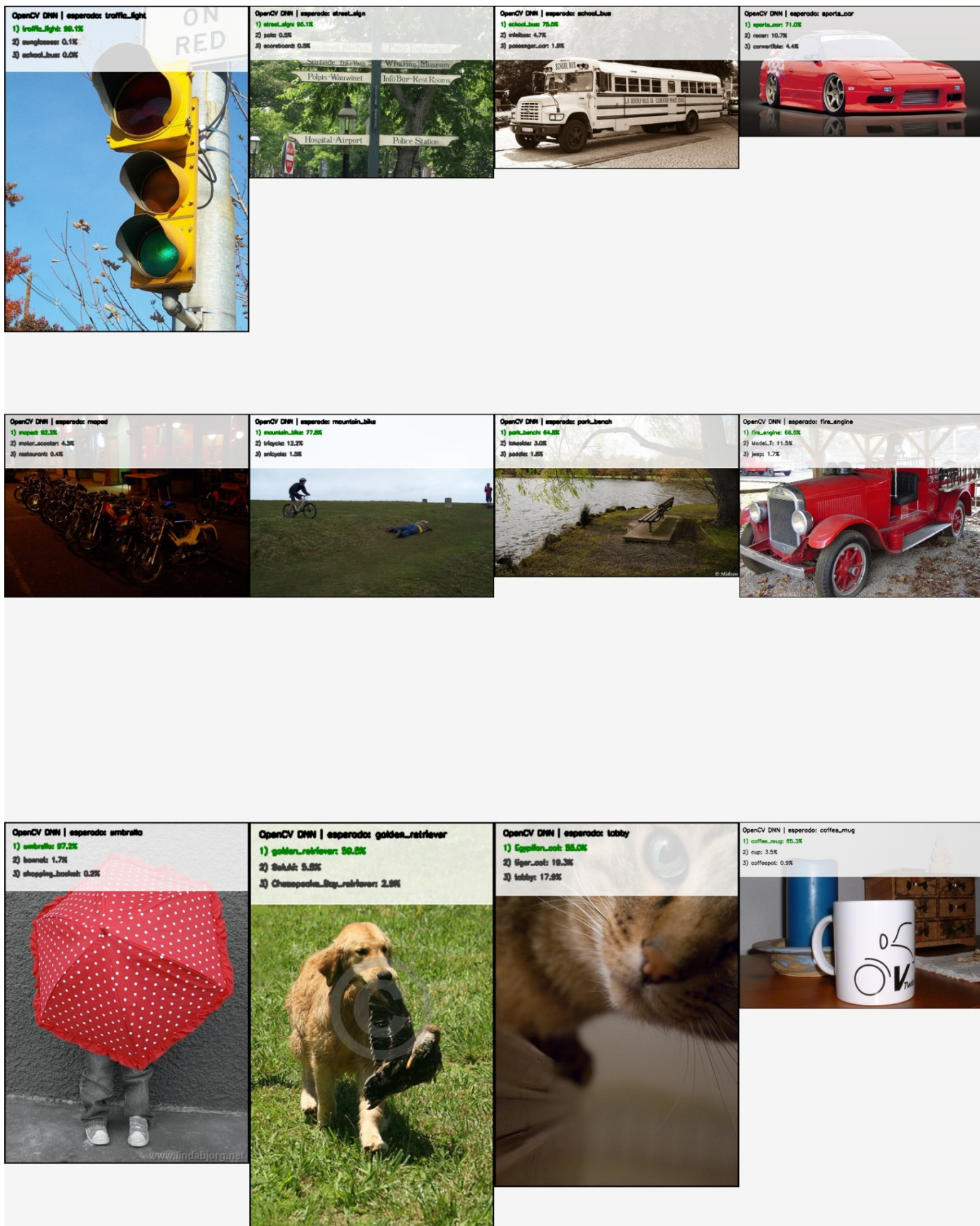


Figura 5 — Top-3 sobreposto às 12 imagens, com a predição do OpenCV DNN em cada quadro

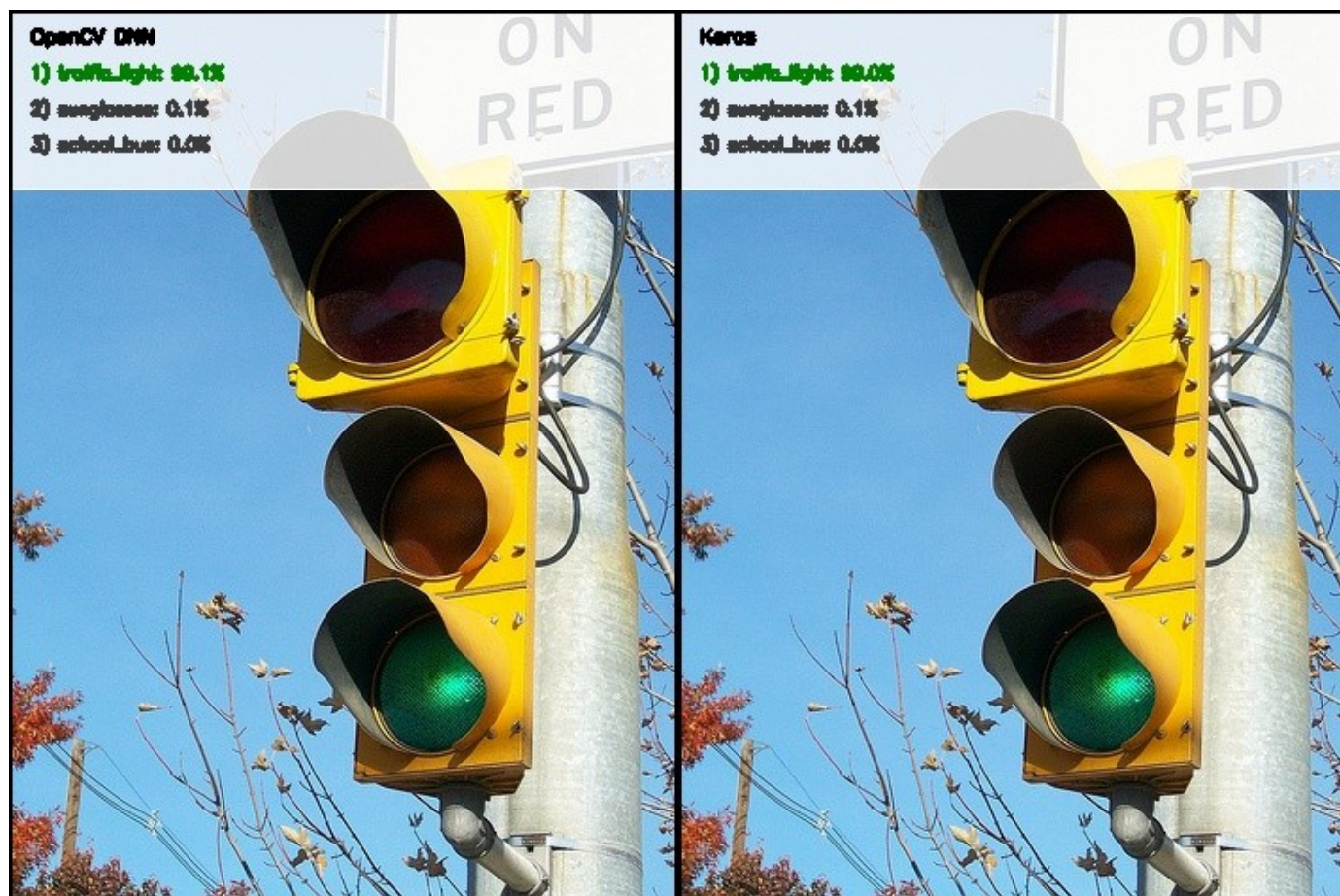


Figura 6 — Comparação lado a lado dos dois backends na mesma imagem: mesma classe no topo, confianças ligeiramente diferentes

3.4 Resultados medidos

Backend	Latência	Memória do processo	Top-1	Top-3	Modelo em disco
OpenCV DNN (TFLite)	7,06 ms	81,8 MB	91,7% (11/12)	100%	13,35 MB
Keras / TensorFlow	13,87 ms	437,0 MB	91,7% (11/12)	100%	14,05 MB

Mesma rede, mesmos pesos, mesma acurácia: o OpenCV DNN é **2x mais rápido** e usa **5,3x menos memória**. A escolha do backend é, portanto, decisão de engenharia, não de qualidade de predição.

Quando preferir o módulo DNN do OpenCV em sistemas embarcados: pegada de memória em dezenas de MB contra centenas; dependência única, já que a mesma biblioteca faz captura, correção, filtros e inferência; nenhuma cópia entre mundos, porque o frame já é uma matriz do OpenCV; e troca de acelerador (CPU, OpenCL, CUDA, VPU) em duas linhas. O Keras continua necessário para treinar, ajustar e experimentar — a regra prática adotada é treinar no Keras e **executar** no OpenCV DNN.

3.5 O que descobrimos no item A

Um erro silencioso de softmax duplo. A MobileNetV2 já termina com uma camada Softmax, preservada na conversão para TFLite, então `net.forward()` devolve probabilidades que já somam 1. Aplicar softmax outra vez, como fazem exemplos genéricos de OpenCV DNN, não altera o ranking das classes nem a acurácia — apenas achata as confianças, derrubando o topo de 99,1% para 0,3%. Em um sistema que decide por limiar, isso quebraria tudo sem gerar erro.

As confianças dos dois backends não são idênticas. A divergência chega a 4,77 pontos percentuais (ônibus escolar: 75,5% contra 70,7%), efeito de implementações distintas dos mesmos operadores.

Consequência prática: um limiar de confiança calibrado no framework de treino precisa ser revalidado no runtime de produção.

O único erro foi **tabby** classificado como **Egyptian_cat**, nos dois backends — duas raças de gato rajado, com o rótulo correto em segundo lugar. Por isso o top-3 é 100%: o erro é de granularidade do vocabulário, não de percepção.

3.6 Item B — pipeline integrado

O notebook [ex2b.ipynb](#) encadeia uma técnica de cada etapa da disciplina sobre um frame do vídeo `vtest.avi` (cena de campus com pedestres, cones de sinalização e veículos), medindo o tempo de cada uma: **undistort** (Exercício 1) → **ROI por cor em HSV** (TP1) → **descritores ORB** (TP2) → **HOG+SVM e Haar Cascade** (TP3) → **classificação com MobileNetV2** (Exercício 2A).

Como o vídeo já vem retificado, aplicamos ao frame a distorção da câmera calibrada e em seguida a corrigimos, simulando o caminho real "lente barata → undistort". A matriz K é reescalada para a resolução do vídeo, porque f_x , f_y , c_x e c_y são medidos em pixels.



Figura 7 — Frame final com as cinco etapas sobrepostas: ROIs de cor em amarelo, pontos-chave ORB em magenta, pessoas detectadas pelo HOG em verde e, no rodapé, a classificação de cada ROI

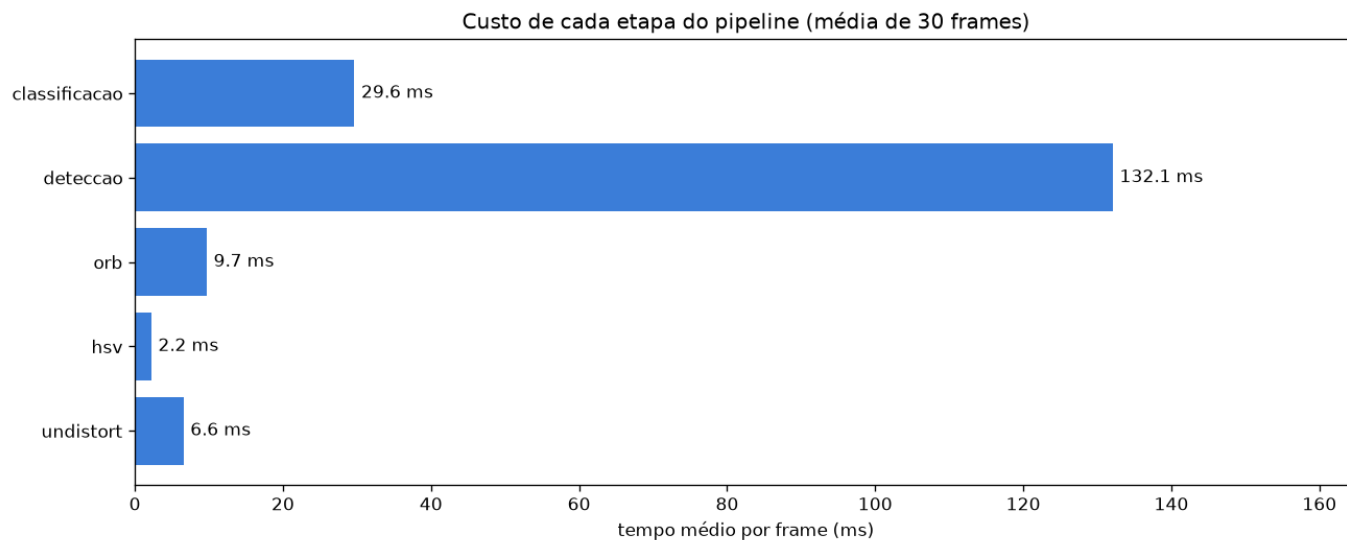


Figura 8 — Custo de cada etapa do pipeline, média de 30 frames

Etapa	Tempo médio	% do total
1. undistort	6,45 ms	3,6%
2. HSV + morfologia	2,28 ms	1,3%
3. ORB	9,45 ms	5,3%
4. HOG + Haar	130,43 ms	72,8%
5. classificação DNN	30,51 ms	17,0%
Total	179,11 ms	5,6 FPS

3.7 O que descobrimos no item B

O gargalo é o detector clássico, com 72,8% do tempo, por motivo estrutural: o HOG varre a imagem com janela deslizante em oito escalas, e cada janela exige calcular o descritor e avaliar o SVM. Otimizar o `undistort`, que custa 3,6%, não muda nada; trocar o detector muda tudo.

A segmentação por cor descartou 81% das regiões candidatas (32 caíram para 6) por filtros geométricos. Sem esse conhecimento externo da cena, o tijolo do prédio e os caixilhos das janelas entram como se fossem cones de sinalização.

O Haar não encontrou nenhum rosto e o classificador devolveu classes sem nexos nos recortes de pedestre (`tricycle` 33%, `broom` 24%). Não é falha de configuração: o Haar exige rosto frontal com resolução mínima, e aqui a pessoa está a cerca de 40 metros; e o ImageNet não possui a classe "pedestre", então a rede devolve a categoria visualmente mais próxima do seu vocabulário. É exatamente o que motiva o exercício seguinte, com modelos treinados em COCO, que tem `person`, `car`, `bicycle` e `traffic light`.

4. Exercício 3 — Detecção com YOLO e SSD e rastreamento com ID

Competências 3.1, 3.2 e 4.3 · Arquivos: [preparar_modelos_ex3.py](#), [detectores.py](#), [ex3a.py](#), [ex3b.py](#)

4.1 Item A — o que o enunciado pedia

Implementar detecção em tempo real com YOLOv4-tiny e SSD MobileNetV2, aplicar Non-Maximum Suppression com limiar 0,4, medir FPS e latência, comparar em tabela com número de parâmetros e tamanho em disco, e concluir qual é mais adequado a robótica embarcada.

4.2 Como resolvemos

Os dois modelos rodam no mesmo backend ([cv2.dnn](#) em CPU), com os mesmos limiares. Usar Ultralytics/PyTorch para o YOLO mediria também a diferença entre runtimes, e não entre arquiteturas.

Cada modelo foi medido em uma passada isolada sobre os mesmos 300 frames. A primeira versão alternava as duas redes no mesmo laço, e o YOLO marcou 141 ms de inferência; sozinho, nos mesmos frames, marcou 36,7 ms. Dois grafos grandes vivos ao mesmo tempo disputam cache e o pool de threads — efeito que, por si só, já é um resultado relevante para quem pensa em rodar dois modelos concorrentes na CPU de um robô.

4.3 Evidências



Figura 9 — YOLOv4-tiny: caixas com classe e confiança, contador de objetos, latência e FPS no cabeçalho



Figura 10 — SSD MobileNetV2 no mesmo frame

Vídeos completos em [evidencias/ex3a_yolov4tiny.mp4](#) e [evidencias/ex3a_ssd.mp4](#).

4.4 Resultados medidos

Modelo	Entrada	FPS médio	FPS no p95	Inferência	Parâmetros	Arquivo
YOLOv4-tiny	416×416	24,6	22,3	36,7 ms	6.062.826	23,1 MB
SSD MobileNetV2	300×300	31,1	27,7	30,6 ms	16.876.287	66,6 MB

Confiança média: YOLO **80,4%**, SSD 72,3%. Detecções por frame: YOLO 6,98, SSD 7,74. Concordância entre os dois modelos (mesma classe, IoU $\geq 0,5$): **87,1%**.

Sensibilidade do YOLO à resolução de entrada, o parâmetro mais direto de ajuste em hardware embarcado:

Entrada	Inferência	FPS	Objetos por frame
256×256	22,4 ms	44,6	0,72
320×320	31,3 ms	31,9	5,22
416×416	48,8 ms	20,5	7,05
512×512	64,6 ms	15,5	7,55

4.5 Conclusão do item A

A recomendação é **calculada pelo próprio script**, a partir de um critério explícito: diferenças de FPS abaixo de 1,5× são recuperáveis ajustando resolução de entrada ou quantizando, logo não decidem o projeto. Medimos 1,26×. No que sobra, o YOLOv4-tiny ganha: **43,5 MB a menos** em disco, 10,8 milhões de parâmetros a menos, confiança média maior (80,4% contra 72,3%) e resolução de entrada ajustável em tempo de execução, sem reverter o modelo. O SSD MobileNetV2 continua preferível quando o alvo é grande e próximo, ou quando o dispositivo já executa TensorFlow Lite quantizado.

Ressalva metodológica: FPS medido em desktop não se transfere para o alvo embarcado; a medição precisa ser repetida na placa final, porque a razão entre os modelos muda com o conjunto de instruções e com o acelerador disponível.

4.6 O que descobrimos no item A

O SSD não detectava nada. O grafo congelado da TensorFlow Object Detection API carrega o próprio sub-grafo `Preprocessor`, que já normaliza os pixels. Aplicando também a normalização no `blobFromImage`, como manda o tutorial genérico, a imagem era normalizada duas vezes: a confiança máxima ficava em 0,058 e nenhuma detecção passava do limiar. Com pixels crus em 0-255, a confiança máxima no mesmo frame foi 0,934.

Mais detecções não significa melhor. O SSD detectou 7,74 objetos por frame contra 6,98 do YOLO, mas sem anotação de referência não há como saber se é recall maior ou falso positivo. Por isso medimos a concordância entre os dois modelos, que delimita o núcleo de objetos confirmados por ambos.

4.7 Item B — rastreamento com ID persistente

O detector é sem memória: cada frame devolve caixas sem vínculo com o anterior. Implementamos o núcleo do algoritmo SORT, sem filtro de Kalman: a cada frame, calcula-se a IoU entre cada trilha ativa e cada detecção nova, casam-se os pares de maior IoU acima de 0,30, detecção sem par vira trilha nova e trilha sem detecção acumula falhas até ser encerrada. Duas salvaguardas: uma trilha só é confirmada após 3 frames (evita contar falso positivo isolado) e sobrevive a 8 frames sem detecção (evita perder o ID quando a pessoa passa atrás de um poste).

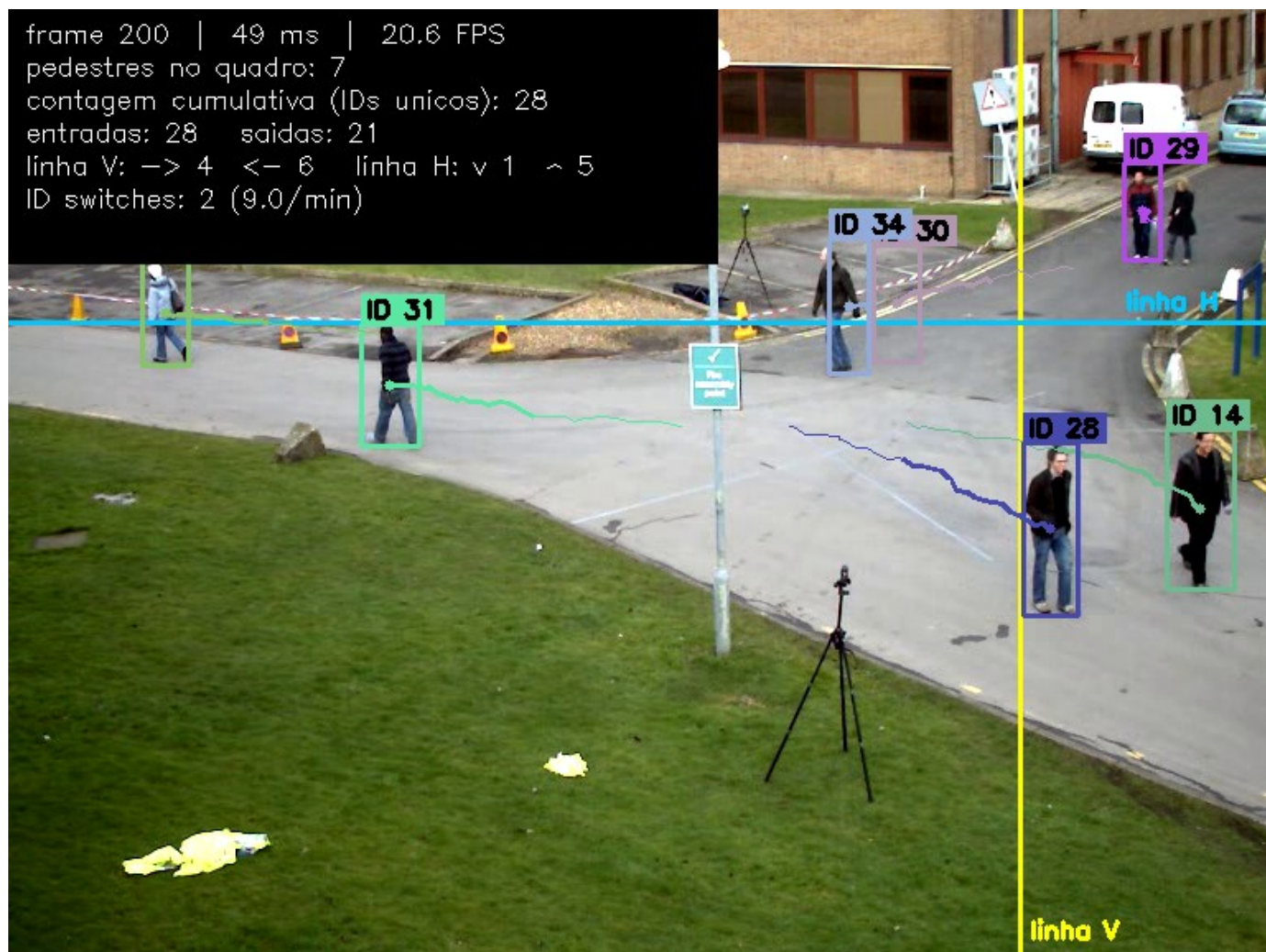


Figura 11 — Rastreamento: caixa e trilha coloridas por ID, trilha dos últimos 30 frames, duas linhas virtuais de contagem e painel com contagem cumulativa, entradas, saídas, cruzamentos e taxa de ID switches

Vídeo de demonstração completo: [evidencias/ex3b_rastreamento.mp4](#) (795 frames).

Métrica	Valor
Frames processados / duração	795 / 53,0 s
Pedestres por frame	5,97 (máximo 11)
Contagem cumulativa de IDs únicos	98
Entradas / saídas detectadas	98 / 91
Cruzamentos das duas linhas virtuais	78
ID switches	9 → 10,19 por minuto
FPS (detecção + rastreamento)	27,6
Trilhas com menos de 1 s de vida	16 de 98 (16%)

Como a taxa de ID switches foi estimada. Não existe anotação de identidade para este vídeo, então a métrica formal do padrão MOT é impossível. Usamos um estimador por reaparecimento: conta-se um switch quando uma trilha é encerrada e, poucos frames depois, nasce outra com $\text{IoU} \geq 0,30$ em relação à última caixa da trilha morta. Os dois vieses estão declarados no código: **subestima** a troca entre duas pessoas que se cruzam, porque aí nenhuma trilha morre, e **superestima** quando alguém sai de cena e outro entra pelo mesmo ponto. O número deve ser lido como taxa de fragmentação, útil para comparar configurações do próprio rastreador.

Limitação conhecida da associação só por IoU: sem modelo de movimento, a associação depende de as caixas se sobreporem entre frames consecutivos. É o que o filtro de Kalman do TP3 resolve, ao comparar a detecção com a **previsão** da posição em vez da última posição observada.

4.8 Aplicação em drone de vigilância urbana e considerações éticas

Tecnicamente, o pipeline deste item é o que um drone de monitoramento executaria. As métricas indicam o que precisaria mudar: a contagem cumulativa herda a fragmentação do rastreador, e por isso superestima pessoas únicas — para fluxo, a contagem por cruzamento de linha é mais robusta; a câmera em movimento enfraquece a premissa de sobreposição entre frames, exigindo compensação por homografia ou filtro de Kalman; e a vista aérea muda a aparência do pedestre, degradando um detector treinado em fotos ao nível do solo.

Quanto à ética, contar pessoas e identificar pessoas são coisas diferentes, e a fronteira é decisão de projeto:

1. **Finalidade e proporcionalidade.** Contagem agregada para dimensionar calçada, semáforo ou transporte é finalidade legítima; a mesma câmera com reconhecimento facial vira vigilância individual. A LGPD exige finalidade específica, informada, e minimização.
 2. **Minimização por construção.** O sistema pode nunca gravar imagem: processa o frame em memória, incrementa contadores e o descarta. O que persiste é "12 pedestres cruzaram para leste entre 14h e 15h". Onde a imagem for necessária, anonimizar na borda antes de qualquer armazenamento.
 3. **Não reidentificação.** O ID vale para um trecho de vídeo e não deve religar a mesma pessoa entre dias ou entre câmeras — esse cruzamento transforma contagem em rastreo de rotina individual.
 4. **Viés e erro desigual.** Detectores erram mais para pele escura, crianças, cadeirantes e pessoas com carrinho; em contagem, isso vira sub-representação sistemática no dado que orienta política pública. Auditar a taxa de detecção por subgrupo é parte do trabalho de engenharia.
 5. **Transparência.** Em espaço público não há consentimento individual viável, o que aumenta o dever de sinalizar, publicar finalidade, prazo de retenção e responsável.
 6. **Uso secundário.** O risco maior está no depósito de dados, não no algoritmo: um histórico de trajetórias coletado para "otimizar o trânsito" é atraente para outros fins. Retenção curta, agregação na origem e registro de acesso são as barreiras técnicas.
 7. **Efeito inibidor.** Vigilância aérea persistente muda o comportamento de quem circula, inclusive o direito de manifestação; a decisão de sobrevoar um bairro não é apenas técnica.
-

5. Exercício 4A — Segmentação semântica comparada à segmentação por cor

Competência 4.4 · Arquivos: [preparar_dados_ex4.py](#), [ex4a.py](#)

5.1 O que o enunciado pedia

Implementar segmentação semântica com DeepLabV3 ou FCN-ResNet50, processar ao menos 5 cenas externas gerando mapa de classes colorido e máscara semitransparente, imprimir a porcentagem de área por classe, comparar lado a lado com a segmentação por cor HSV do TP1 e discutir vantagens e limitações de cada abordagem para um veículo autônomo.

5.2 Como resolvemos

Rodamos **as duas arquiteturas**, DeepLabV3-ResNet50 e FCN-ResNet50, porque a divergência entre elas na mesma imagem é, por si, um indicador de confiabilidade. Processamos **7 cenas**: três frames do [vtest.avi](#) e quatro fotografias reais já usadas no Exercício 2A, para não concluir nada a partir de uma única cena.

A faixa HSV do asfalto não foi arbitrada: **amostramos os pixels** nos próprios frames. A via tem saturação entre 15 e 20 e brilho entre 136 e 200; a grama tem saturação 197, e por isso é excluída; a calçada tem saturação 55 e brilho 167, e por isso **entra junto com a via**.

5.3 Evidências



Figura 12 — Painel 2×2 da cena de via: original, DeepLabV3, FCN e a segmentação por cor do TP1. A rede identifica os pedestres e classifica todo o resto como fundo; o HSV encontra a área navegável, mas não separa a via da calçada



Figura 13 — Máscara semântica semitransparente sobre a cena do ônibus escolar, a única classe de veículo presente no vocabulário do VOC

5.4 Resultados medidos

Cena	Pixels como "fundo"	Classe útil	Via pelo HSV
campus_gramado	96,7%	3,3%	31,0%
campus_passeio	98,1%	1,9%	31,3%
campus_via	97,6%	2,4%	30,8%
park_bench	100,0%	0,0%	14,9%
school_bus	64,8%	35,2%	7,7%
street_sign	100,0%	0,0%	0,8%
traffic_light	100,0%	0,0%	3,4%

Custo por imagem	DeepLabV3	FCN	HSV
campus_via	985 ms	836 ms	3,9 ms (253× mais rápido)
school_bus	947 ms	659 ms	1,2 ms (785× mais rápido)

Memória: 538 MB com os dois modelos carregados, contra 81,8 MB do classificador no OpenCV DNN. Concordância entre DeepLabV3 e FCN: 99,4% dos pixels nas cenas do campus e 91,9% na foto do ônibus.

5.5 O que descobrimos

Em média, 93,9% dos pixels foram rotulados como "fundo". Os pesos disponíveis no torchvision foram treinados no vocabulário do PASCAL VOC, que tem 20 classes de objeto e **não contém pista, calçada, faixa de pedestre, poste, prédio nem vegetação.** Numa cena de rua, portanto, o asfalto é necessariamente "fundo" — não por erro do modelo, mas porque a classe não existe para ele. Banco de praça, placa de rua e semáforo deram 100% de fundo pelo mesmo motivo; o ônibus escolar deu 35,2% de classe útil porque **bus** está no vocabulário. É por isso que veículos autônomos reais usam modelos

treinados em Cityscapes, BDD100K ou Mapillary, cujos vocabulários incluem **road**, **sidewalk**, **lane marking**, **pole** e **traffic sign**.

As duas abordagens falham em pontos opostos, e é isso que as torna complementares. A cor é determinística, auditável por seis números e roda em poucos milissegundos, mas não tem noção de objeto: asfalto e concreto são ambos acinzentados, então via e calçada se confundem. A rede decide por contexto e forma, separa um pedestre de roupa escura sobre asfalto escuro e entrega fronteira por pixel, mas custa duas a três ordens de grandeza mais, tem vocabulário fechado e **falha em silêncio**, entregando máscara plausível mesmo fora do domínio de treino.

Para um veículo autônomo, a divisão de papéis é por criticidade e prazo: segmentação semântica como camada principal de compreensão de cena, rodando em acelerador; técnicas clássicas de cor como camada rápida e verificável para alvos de cor normatizada (cone de obra, faixa amarela, luz de freio); detecção por caixas para agentes que exigem rastreamento; e, acima de tudo, redundância — divergência forte entre as camadas é sinal utilizável para reduzir velocidade, em vez de confiar em uma máscara que pode estar plausivelmente errada.

6. Exercício 4B — Relatório integrativo

Competência 4.4 e todas as anteriores · Arquivo: [relatorio_integrativo.md](#) (entregue também como arquivo próprio)

Esta seção é a íntegra do relatório integrativo exigido no item B do Exercício 4: 2.397 palavras e as cinco seções obrigatórias — diagrama do pipeline, tabela comparativa com métricas reais, análise de viabilidade em 5 W, proposta de arquitetura de percepção e as três lacunas para a DR4.

6.1 Diagrama do pipeline completo

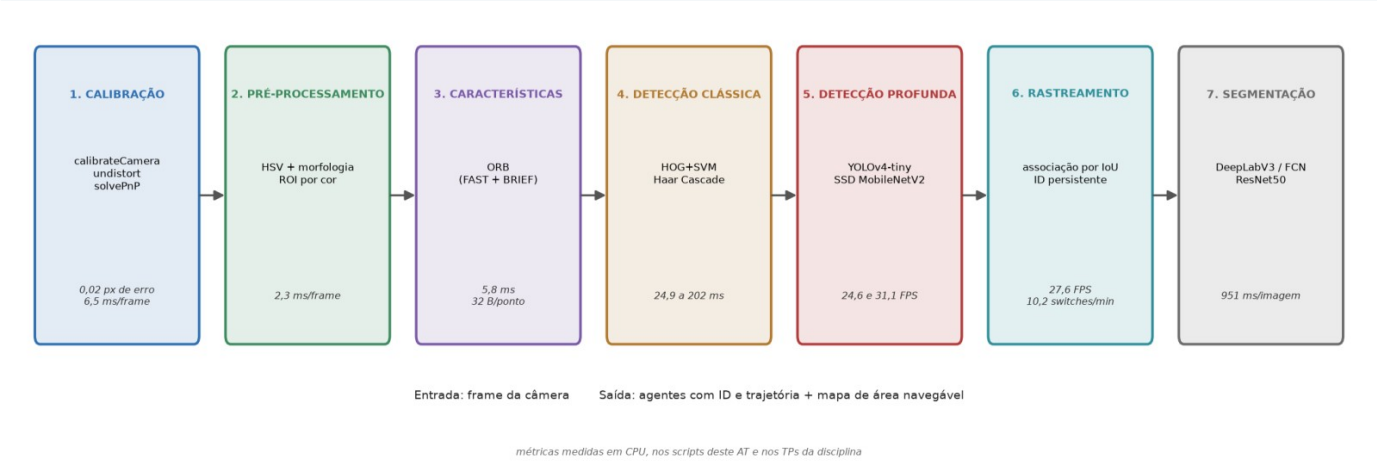
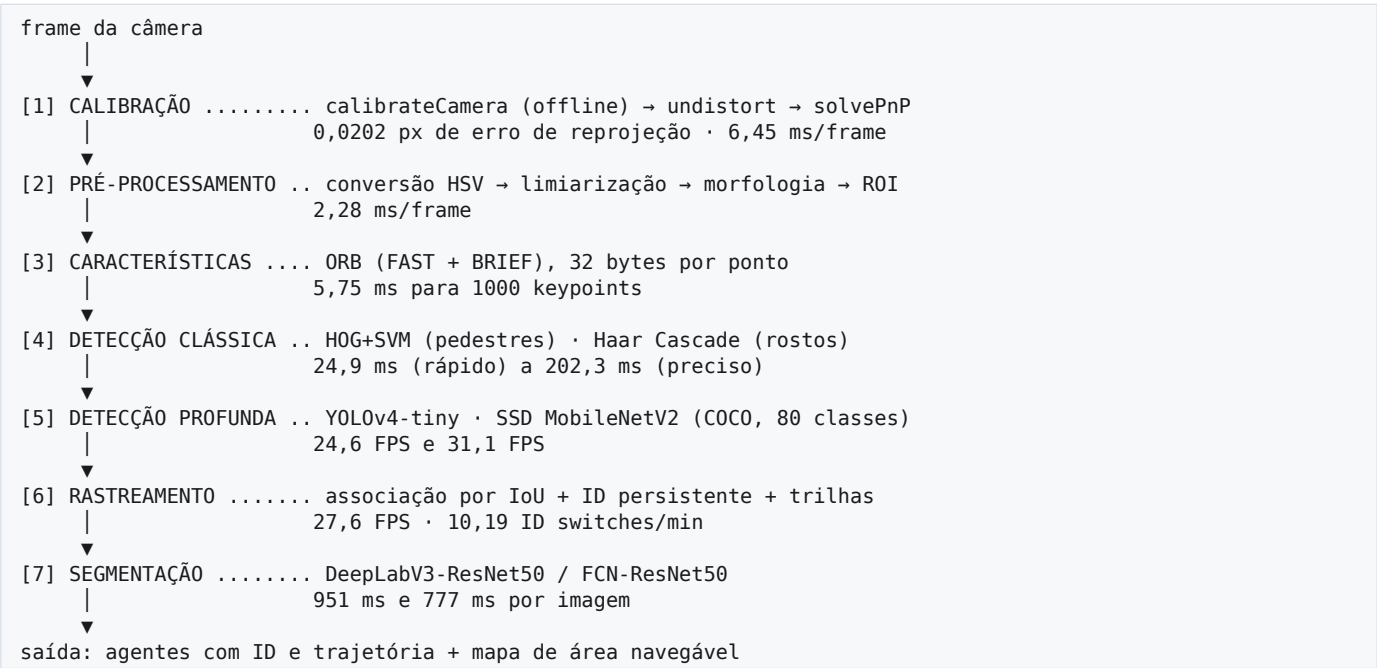


Figura 14 — Pipeline completo de percepção visual



O fluxo é sequencial no diagrama por clareza didática, mas a dependência real é parcial. A calibração é pré-requisito de tudo que envolva medição métrica, pois define a relação entre pixel e ângulo. Já as etapas 2 a 7 consomem o mesmo frame retificado e poderiam rodar em paralelo, em processos ou aceleradores distintos — foi justamente a tentativa de executar duas redes intercaladas na mesma CPU que inflou a latência do YOLO de 36,7 ms para 141 ms no Exercício 3A, resultado que motivou medir cada modelo em passada isolada.

6.2 Tabela comparativa das técnicas

Recorte das técnicas mais relevantes; a tabela completa, com 27 entradas, está em [evidencias/ex4b_metricas.md](#).

Etapa	Técnica	Tempo	Qualidade medida	Complexidade	Fonte
Pré-processamento	Segmentação HSV + morfologia	2,28 ms	81% das regiões candidatas descartadas por filtros geométricos	O(n) por pixel, sem treino	AT ex2b
Pré-processamento	HSV da via (asfalto)	3,4 ms	30,8% da imagem como navegável; não separa via de calçada	O(n) por pixel	AT ex4a
Calibração	<code>calibrateCamera</code> (24 imagens)	offline	erro de reprojeção 0,0202 px; erro de fx 0,078%	uma vez por câmera	AT ex1a
Calibração	<code>undistort</code> por frame	6,45 ms	corrige até 27 px de deslocamento radial na borda	O(n) por pixel	AT ex2b
Pose	<code>solvePnP</code> + <code>projectPoints</code>	0,27 ms	erro de translação 0,31 mm; rotação 0,028°	iterativo, 42 pontos	AT ex1b
Características	ORB (1000 keypoints)	5,75 ms	descritor binário de 32 bytes	FAST + BRIEF	TP2 ex3a
Características	AKAZE	29,4 ms	2666 keypoints, 61 bytes	difusão não linear	TP2 ex3a
Características	SIFT	52,4 ms	2419 keypoints, 128 floats	pirâmide DoG	TP2 ex3a
Detecção clássica	HOG+SVM (rápido)	24,9 ms	1,39 detecção/frame	janela deslizante	TP3 ex1a
Detecção clássica	HOG+SVM (preciso)	202,3 ms	4,22 detecções/frame	pirâmide fina	TP3 ex1a
Detecção clássica	HOG+SVM treinado por nós	~5800 ms	precisão 0,89 e F1 0,64	treino próprio	TP3 ex1b
Detecção clássica	Haar Cascade (rostos)	—	0 rostos em cena a 40 m	cascata	AT ex2b
Subtração de fundo	MOG2	4,48 ms	4,28 objetos/frame (223 FPS)	mistura por pixel	TP3 ex2a
Subtração de fundo	KNN	3,66 ms	4,39 objetos/frame (273 FPS)	vizinhos por pixel	TP3 ex2a
Classificação	CNN no MNIST	—	99,41% com 421.642 parâmetros	2 blocos conv.	TP3 ex3a
Classificação	MLP no MNIST	—	98,00% com 535.818 parâmetros	2 densas	TP3 ex3a
Classificação	MobileNetV2 (OpenCV DNN/TFLite)	7,06 ms	91,7% top-1 e 100% top-3; 81,8 MB de RAM	3.538.984 parâmetros	AT ex2a

Etapa	Técnica	Tempo	Qualidade medida	Complexidade	Fonte
Classificação	MobileNetV2 (Keras/TensorFlow)	13,87 ms	mesma acurácia; 437,0 MB de RAM	mesma rede	AT ex2a
Detecção profunda	YOLOv4-tiny 416	40,6 ms	24,6 FPS; confiança média 80,4%; 23,1 MB	6.062.826 parâmetros	AT ex3a
Detecção profunda	YOLOv4-tiny 320	31,3 ms	31,9 FPS; 5,22 objetos/frame	mesma rede	AT ex3a
Detecção profunda	SSD MobileNetV2 300	32,2 ms	31,1 FPS; confiança média 72,3%; 66,6 MB	16.876.287 parâmetros	AT ex3a
Rastreamento	IoU + ID persistente	36,3 ms	27,6 FPS; 98 IDs; 10,19 switches/min	associação gulosa	AT ex3b
Rastreamento	CamShift	—	perdeu o alvo no frame 275	média deslocada	TP3 ex2a
Segmentação	DeepLabV3-ResNet50	951 ms	93,9% dos pixels caem em "fundo"	ResNet50 + ASPP	AT ex4a
Segmentação	FCN-ResNet50	777 ms	99,4% de concordância com o DeepLabV3	ResNet50 + FCN	AT ex4a

Três leituras transversais merecem destaque.

Velocidade e valor não andam juntos. O KNN de subtração de fundo é a técnica mais rápida de todas (3,66 ms, 273 FPS) e, ao mesmo tempo, a que menos informa: separa "mexeu" de "não mexeu", sem dizer o que mexeu. A segmentação semântica é 260 vezes mais lenta e devolve rótulo por pixel. O custo, aqui, é o preço da semântica.

A mesma rede tem dois custos diferentes. MobileNetV2 idêntica, com os mesmos pesos, roda em 7,06 ms e 81,8 MB pelo OpenCV DNN, ou 13,87 ms e 437,0 MB pelo Keras. A escolha do runtime, portanto, é decisão de engenharia e não de acurácia: a concordância de top-1 entre os dois foi de 12/12. Mesmo assim, as confianças divergiram em até 4,77 pontos percentuais, o que significa que **um limiar calibrado no framework de treino precisa ser revalidado no runtime de produção**.

Técnica clássica não é sinônimo de barata. O HOG+SVM com janela deslizante treinado no TP3 levou cerca de 5,8 s por imagem, e o HOG "preciso" do detector pronto, 202,3 ms. O YOLOv4-tiny, uma rede profunda, resolve o mesmo problema em 40,6 ms com qualidade maior. No Exercício 2B, o HOG respondeu por 72,8% do tempo de um pipeline de cinco etapas.

6.3 Viabilidade em hardware embarcado com restrição de 5 W

O orçamento de 5 W corresponde a uma Raspberry Pi 4, a um Jetson Nano em modo 5 W ou a uma placa com SoC ARM de quatro núcleos sem acelerador dedicado. Duas correções são necessárias antes de transportar as medições:

1. **Fator de desconto de CPU.** As medições deste AT foram feitas em CPU x86 de desktop. Uma CPU ARM de 5 W costuma ser de 5 a 10 vezes mais lenta em carga de convolução, por menor frequência, menos núcleos úteis e ausência de AVX. Adoto o fator conservador de 8×.
2. **Orçamento de prazo.** Para reação em trânsito urbano a 30 km/h, o veículo percorre 8,3 m/s; um atraso de 100 ms equivale a 83 cm de deslocamento sem informação nova. Fixo então 100 ms como prazo do laço de percepção.

Aplicando o fator 8× ao tempo medido ([evidencias/ex4b_grafico_custo.png](#)):

Cabem no prazo (9 técnicas). HSV + morfologia (2,28 ms → 18 ms), HSV da via (3,4 ms → 27 ms), `undistort` (6,45 ms → 52 ms), `solvePnP` (0,27 ms → 2 ms), ORB (5,75 ms → 46 ms), MOG2 (4,48 ms → 36 ms), KNN (3,66 ms → 29 ms), Caffe gender_net (8,19 ms → 66 ms) e MobileNetV2 pelo OpenCV DNN (7,06 ms → 56 ms). Observe que nenhuma delas isolada esgota o prazo, mas a soma esgota: o pipeline do Exercício 2B, com 179,11 ms medidos, iria a 1,43 s — ou seja, **a composição é o problema, não cada peça**.

Ficam no limite (3 técnicas). YOLOv4-tiny a 320×320 (31,3 ms → 250 ms), SSD MobileNetV2 (32,2 ms → 258 ms) e YOLOv4-tiny a 416×416 (40,6 ms → 325 ms). Em CPU pura, portanto, o detector profundo entrega de 3 a 4 FPS. Viabilizá-lo exige quantização int8, que tipicamente reduz o custo em 2 a 4 vezes e o tamanho em 4 vezes, ou um acelerador: uma NPU de 2 TOPS dentro do mesmo envelope de 5 W recoloca o YOLOv4-tiny acima de 20 FPS.

Não cabem (4 técnicas). HOG preciso (202,3 ms → 1,6 s), HOG+SVM próprio (5,8 s → 46 s), FCN-ResNet50 (777 ms → 6,2 s) e DeepLabV3-ResNet50 (951 ms → 7,6 s). A memória agrava o caso da segmentação: os dois modelos somaram 538 MB de RAM, contra 81,8 MB do classificador no OpenCV DNN. Em placa de 1 GB compartilhado com o sistema, é inviável. O caminho é trocar o backbone ResNet50 por MobileNetV3 ou por uma arquitetura desenhada para borda (BiSeNet, DDRNet, SegFormer-B0), reduzir a resolução de entrada e quantizar.

Conclusão de projeto: **em 5 W, o laço rápido é clássico e o laço semântico é lento e assíncrono**. A calibração, a limiarização por cor, os descritores ORB e a subtração de fundo cabem folgadoamente a 10 Hz. O detector profundo cabe se quantizado ou acelerado. A segmentação semântica densa só cabe em versão leve e executada a taxa reduzida, sem bloquear o laço de controle.

6.4 Arquitetura de percepção para um veículo autônomo urbano

A proposta integra cinco técnicas da disciplina em três laços com prazos distintos, porque o erro de projeto mais comum é submeter tudo à mesma taxa.

Camada 0 — calibração e retificação (offline + 10 Hz). `calibrateCamera` executado uma vez por câmera, com o padrão cobrindo a periferia do quadro. O Exercício 1A mostrou por que a cobertura importa: com 18 poses centrais, o erro de reprojeção ficou em 0,0182 px, aparentemente ótimo, mas o modelo de distorção errava 27,5 px na borda da imagem; com 6 poses periféricas, o erro na borda caiu para 0,24 px. Em operação, o `undistort` vira `remap` com mapas pré-calculados na inicialização.

Camada 1 — laço rápido, clássico e verificável (10 a 30 Hz). Segmentação HSV para alvos de cor normalizada (cone de obra, faixa amarela, luz de freio, estado do semáforo), morfologia e filtros geométricos. Custo medido de 2,28 ms por frame, determinístico e auditável por seis números. Serve de camada de sanidade: é o que continua respondendo se a rede falhar.

Camada 2 — detecção e rastreamento de agentes (10 a 15 Hz). YOLOv4-tiny quantizado em NPU, com NMS a 0,4, seguido de rastreamento por IoU com ID persistente. Medi 27,6 FPS para detecção mais rastreamento em CPU de desktop, com 10,19 ID switches por minuto e 16% das trilhas vivendo menos de 1 s. Para uso real, esse rastreador precisa do filtro de Kalman do TP3: associar pela **previsão** da posição, e não pela última posição observada, é o que reduz troca de ID em oclusão. A escolha do YOLOv4-tiny sobre o SSD segue os dados do Exercício 3A: apesar de 1,26× mais lento, ocupa 43,5 MB menos em disco, tem 10,8 milhões de parâmetros menos, decide com confiança média maior (80,4% contra 72,3%) e permite trocar resolução em tempo de execução — 320×320 quase dobra o FPS.

Camada 3 — compreensão de cena (1 a 5 Hz, assíncrona). Segmentação semântica leve treinada em Cityscapes, para área navegável, calçada, faixa e guia. O Exercício 4A quantificou por que não serve usar os pesos VOC do torchvision: 93,9% dos pixels das cenas externas foram rotulados como "fundo", porque o vocabulário não tem pista nem calçada. Roda em taxa reduzida, publicando um mapa de área navegável que o laço de controle consome como último valor válido.

Fusão e segurança. O rastreador fornece trajetória e velocidade dos agentes; a segmentação fornece o espaço livre; o laço de cor fornece sinais normalizados. A regra de decisão é conservadora: divergência forte entre camadas — por exemplo, segmentação declarando via livre onde o detector mantém um pedestre com ID estável — não é resolvida por voto, e sim por redução de velocidade, porque a rede falha

silenciosamente, entregando máscara plausível mesmo fora do domínio de treino. Os descritores ORB (5,75 ms) entram como quarta técnica com papel específico: compensar o movimento da câmera por homografia entre frames, o que estabiliza a associação temporal quando a plataforma se desloca.

6.5 Três lacunas a serem endereçadas na DR4

Lacuna 1 — localização e mapeamento (SLAM). Todo este pipeline responde "o que estou vendo", nunca "onde estou". No Exercício 1B estimei a pose de um padrão em relação à câmera com 0,31 mm de erro, mas isso é pose relativa a um alvo conhecido e cooperativo. Falta o problema inverso: estimar a trajetória da própria câmera em ambiente desconhecido, fechar laços para corrigir deriva acumulada e manter um mapa consistente. Os descritores ORB do TP2 são o insumo dessa etapa (daí o nome ORB-SLAM), mas a disciplina parou na correspondência entre duas imagens, sem grafo de poses, sem otimização global e sem tratamento de deriva.

Lacuna 2 — profundidade métrica e fusão de sensores. Todas as saídas aqui são 2D. Uma caixa de 92×183 px não diz se o pedestre está a 8 m ou a 25 m, e sem essa distância não existe tempo até a colisão nem frenagem calculada. Falta profundidade por estéreo calibrado (visto superficialmente no TP1 e no TP2), estimativa monocular aprendida e, principalmente, **fusão com LiDAR, radar e odometria**, incluindo sincronização temporal e transformação entre referenciais. A câmera é o sensor mais informativo e o menos confiável em chuva, neblina, contraluz e noite; nenhuma das medições deste AT foi feita nessas condições.

Lacuna 3 — previsão, decisão e controle com garantia de prazo. O pipeline termina em percepção; não há previsão de trajetória dos agentes, planejamento de rota, nem controle. As trilhas de 30 frames do Exercício 3B são a entrada natural de um preditor, mas prever o movimento de um pedestre exige modelo de intenção, e não extrapolação linear. Junto vem a lacuna de engenharia de tempo real: medi latência média e percentil 95 (por exemplo, 22,3 FPS no pior caso do YOLO), mas não há orçamento de prazo fim a fim, degradação graciosa quando um módulo perde o prazo, nem estratégia de validação de segurança — o que se mede, com que cobertura de cenário e sob qual norma é considerado seguro o suficiente para circular com pessoas ao redor.

7. Conclusão

O AT partiu de quatro enunciados independentes e terminou em um pipeline único, medido ponta a ponta. Três resultados resumem o trabalho:

A métrica padrão pode enganar. No Exercício 1, o erro de reprojeção ficou ligeiramente pior ao acrescentar seis poses periféricas — e mesmo assim o erro do modelo de distorção na borda do quadro caiu de 27,5 px para 0,24 px. Só medir o efeito do modelo sobre todo o quadro revelou isso.

A escolha de runtime é engenharia, não acurácia. A mesma MobileNetV2, com os mesmos pesos, roda em 7,06 ms e 81,8 MB pelo OpenCV DNN ou em 13,87 ms e 437,0 MB pelo Keras, com acurácia idêntica. Em uma placa de 1 GB, essa diferença decide se o programa roda.

O vocabulário do modelo limita o que é possível perceber. No Exercício 4, 93,9% dos pixels das cenas externas foram classificados como "fundo", porque o conjunto de treino não tem pista nem calçada. Nenhum ajuste de código corrige isso: é preciso trocar o conjunto de treino.

O caminho natural, já apontado na seção 6.5, é a DR4: localização e mapeamento, profundidade métrica com fusão de sensores, e previsão, decisão e controle com garantia de prazo.

Repositório

O código-fonte e as evidências em **resolução original** — imagens PNG sem perdas e vídeos na qualidade de gravação — estão publicados em:

<https://github.com/gabriel-fcruzz/sistemas-roboticos-dr2-at>

O pacote entregue contém os mesmos arquivos, com imagens e vídeos recomprimidos para caber no limite de 20 MB da plataforma: JPG com qualidade 90 e largura máxima de 1600 px, e vídeo em H.264 com CRF 28. Gráficos e mapas de classe, que têm poucas cores, foram mantidos em PNG porque nesse caso o formato sem perdas é menor que o JPG.